YTKI: Интелигентна мултиагентна система за мониторинг и реакция при инциденти в облачни услуги

Домашна работа 2: работна програма на проекта

Име и фамилия	Факултетен номер
Йоан Байчев	0MI0600328
Тугай Кафа	4MI0600275
Калоян Марков	4MI0600392
Илиан Запрянов	8MI0600405

2 Описание на работната програма по проекта

2.1 Общо описание

Основна цел. Да изградим и пуснем в употреба система от AI агенти, която денонощно следи алъртите, логовете и health сигналите от всички среди, сама прави първоначалното разследване на всеки инцидент и подава на дежурния инженер готов анализ. Така искаме да съкратим времето за откриване и възстановяване (MTTD и MTTR) и да намалим броя на фалшивите алърти, които стигат до хората.

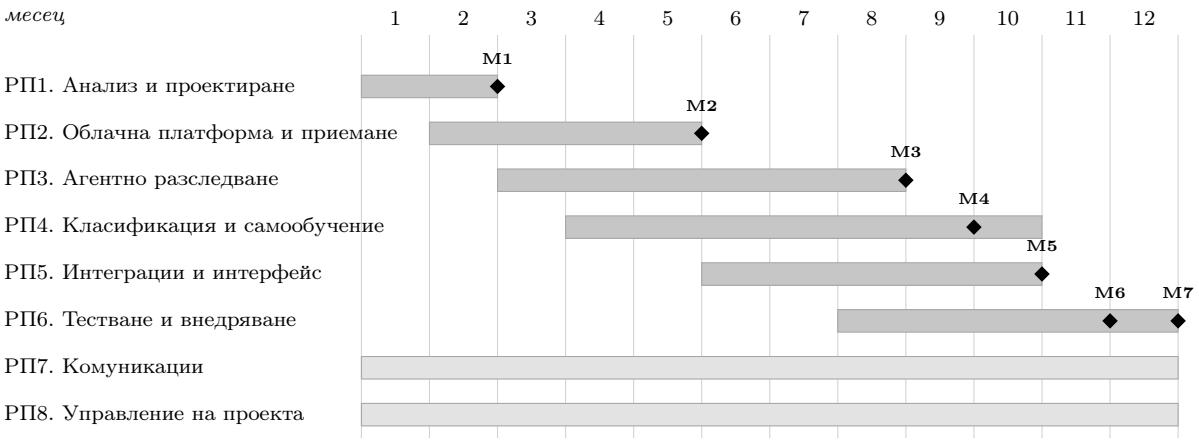
Краен резултат. Работеща в продукция версия 1.0 на YTKI, внедрена в AWS (регион Франкфурт). Тя приема алъртите от dev, staging, int-prod и prod, разследва ги с помощта на агенти, класифицира ги като фалшив алърт, деградация или прекъсване, създава събитие в Splunk и отваря тикет към отговорния екип. Към нея има табло, чат „Ask Alerts“ и документация за потребителите и за поддръжката.

Очакван ефект. По-кратки и по-редки прекъсвания за клиентите, по-малко излишни събуждания на дежурните и проследимост на инцидентите, която улеснява докладването по NIS2.

Подцели и междинни резултати. За да стигнем до крайния резултат, разделяме работата на шест подцели. Всяка от тях е цел на отделен работен пакет и завършва с междинен резултат (milestone), който може да се провери. Към тях се добавят задължителните пакети за комуникации (РП7) и за управление на проекта (РП8).

РП	Подцел	Междинен резултат	Месец
1	Да изясним нуждите, изискванията и ограниченията и да проектираме системата и начина, по който ще я внедрим в AWS	**M1.** Изисквания и архитектура, одобрени от DevOps и SRE екипите	2
2	Да изградим средата в AWS и алъртите от всички среди да постъпват в реално време и в общ формат	**M2.** Алъртите от четирите среди постъпват в YTKI, разгърната в AWS	5
3	Агентите сами да разследват всеки нов алърт	**M3.** За всеки алърт се създава отчет с хипотеза за първопричина	8
4	Класификацията да е достатъчно точна, за да могат дежурните да разчитат на нея	**M4.** F1 поне 0,8 върху тестов набор от минали инциденти	9
5	Резултатите от анализа да стигат автоматично до отговорния екип	**M5.** Алърт минава целия път до събитие в Splunk и тикет	10
6	Системата да бъде проверена в реални условия и пусната в продукция	**M6.** Приключен пилот в сенчест режим в staging и int-prod	11
		M7. YTKI v1.0 работи в prod	12

Продължителност и етапи. Проектът е планиран за 12 месеца. Първите два месеца са за иницииране и планиране. Основната разработка е от втори до десети месец, а от осми до единадесети тестваме и провеждаме пилот. Последният месец е за пускане в продукция и приключване на проекта. Управлението и комуникациите вървят през цялото време. Разпределението на пакетите във времето и междинните резултати са показани на фиг. 1.



Фигура 1. Разпределение на работните пакети във времето и междинни резултати (M1–M7)

2.2 Работна програма и задачи

Йерархична структура на работата

Структурата на работата е организирана по резултати (deliverable-based WBS) и е показана на фиг. 2. На първо ниво работните пакети са групирани в четири направления според експертизата, която изискват: анализ и проектиране (бизнес анализатори и архитект), имплементиране и внедряване (разработчици, QA и DevOps инженери), комуникации и управление на проекта. Всяко направление завършва със собствен резултат, а проектът е готов, когато са готови всички. На второ ниво са работните пакети, а на трето – задачите в тях.

Обща методология

Работата е разделена така, че всеки пакет да събира дейности, за които е нужна сходна експертиза, и да завършва с резултат, който може да се провери. Имплементирането е най-голямото направление, затова го разделяме на пет пакета. Четири от тях следват пътя на един алърт през системата (фиг. 1 от Домашна работа 1): приемане, разследване, класификация и оперативно действие. Петият обхваща тестването и пускането в продукция.

В РП1 работят бизнес анализатор и архитект. В РП2 основната роля е на DevOps и backend инженерите, в РП3 са нужни разработчици с опит с LLM и агентни системи, а в РП4 работи специалист по данни. РП5 е за разработчиците на интеграциите и интерфейса, а РП6 е за QA и SRE инженерите.

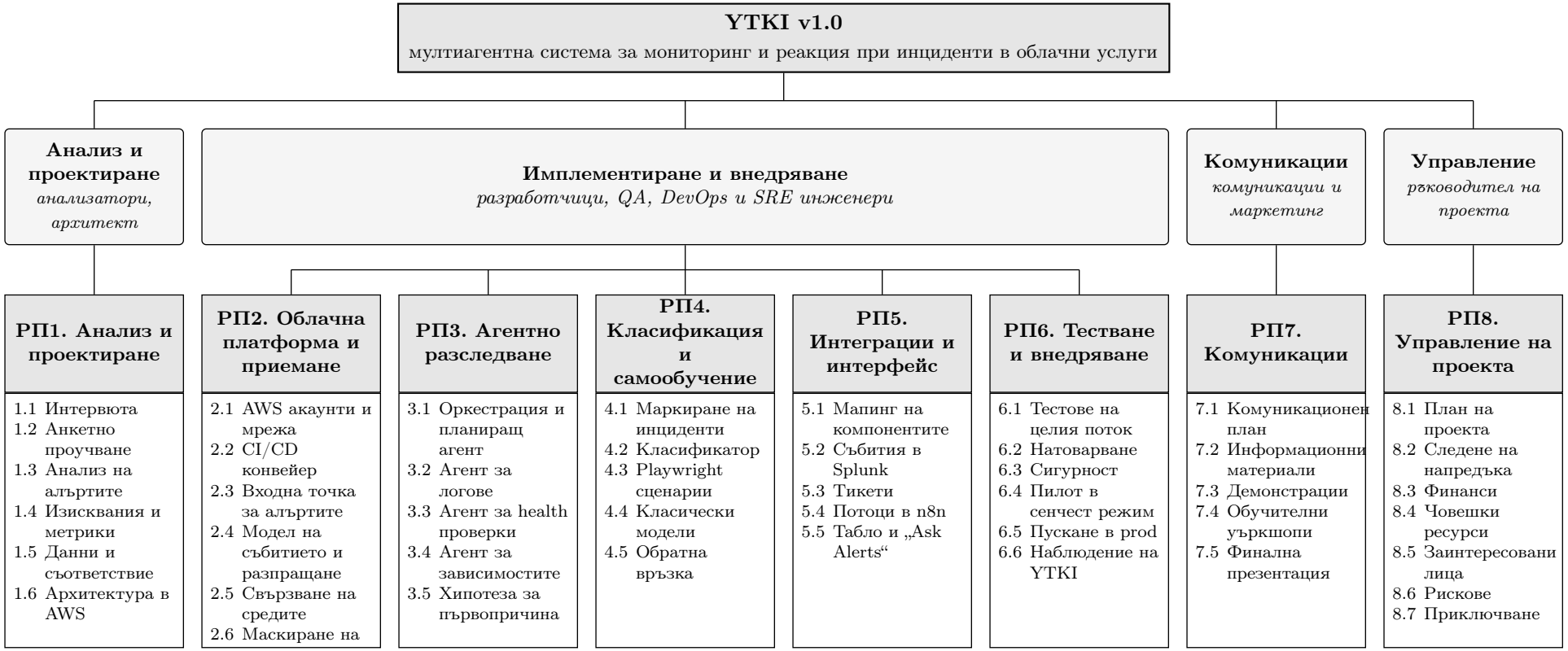
Пакетите от РП2 до РП5 се застъпват във времето. Работим на двуседмични спринтове и всеки пакет може да започне, щом предишният е дал първа работеща версия. Част от РП3 и РП5 стъпва на вече работещия прототип (таблото и „Ask Alerts“), така че там голяма част от работата е разширяване, а не разработка от нулата.

Начинът, по който системата ще бъде внедрена в AWS, е описан в Приложение А. Архитектурата се уточнява в задача 1.6, средата се изгражда в РП2, а пускането в продукция е в РП6.

Обща структура на работната програма

Таблица 1. Работни пакети

Номер	**Име на работен пакет**	**Цел на работен пакет**
Анализ и проектиране		
РП1	Анализ и проектиране	(подцел 1) Изясняване на нуждите на екипите, изискванията, наличните данни и ограниченията и проектиране на системата и на внедряването в AWS
Имплементиране и внедряване		
РП2	Облачна платформа и приемане на данни	(подцел 2) Изграждане на средата в AWS и единен поток в реално време на алъртите, логовете и health сигналите от всички наблюдавани среди
РП3	Агентно разследване	(подцел 3) Автоматично разследване на всеки нов алърт от специализирани агенти
РП4	Класификация и самообучение	(подцел 4) Точна класификация на инцидентите, която се подобрява с натрупването на история
РП5	Интеграции и интерфейс	(подцел 5) Резултатите от анализа да стигат автоматично до отговорния екип и да се виждат на едно място
РП6	Тестване и внедряване	(подцел 6) Проверка на системата в реални условия и пускане в продукция
Комуникации		
РП7	Комуникации	Заинтересованите лица да знаят за проекта, да разбират резултатите и да могат да работят със системата
Управление		
РП8	Управление на проекта	Проектът да приключи в срок, в рамките на бюджета и с нужното качество



Фигура 2. Йерархична структура на работата по проекта (deliverable-based WBS): направления, работни пакети и задачи

Описание на работните пакети

Таблица 2. Описание на работните пакети

Номер и име на работен пакет: РП1: Анализ и проектиране	
Начало (месец): 1	**Край** (месец): 2
Цел: Изясняване на нуждите на екипите, изискванията, наличните данни и ограниченията и проектиране на системата и на внедряването в AWS.	
Основни задачи: 1.1. Интервюта с дежурните инженери и собствениците на услуги 1.2. Анкетно проучване сред DevOps и SRE екипите (брой алърти, фалшиви тревоги, време за реакция) 1.3. Анализ на алъртите и инцидентите, събрани от прототипа 1.4. Изисквания към системата и метрики за успех (MTTD, MTTR, дял на фалшивите алърти) 1.5. Правила за работа с данните и съответствие (GDPR, NIS2, какви данни може да се подават към LLM) 1.6. Архитектура на системата и план за внедряване в AWS (Приложение A)	
Резултати (deliverables): D1.1. Обобщение на интервютата (м. 1) D1.2. Доклад от анкетата (м. 1) D1.3. Доклад за текущия процес с базови стойности на MTTD и MTTR (м. 2) D1.4. Спецификация на изискванията (м. 2) D1.5. Правила за работа с данните (м. 2) D1.6. Архитектурен документ и план за внедряване (м. 2)	

Номер и име на работен пакет: РП2: Облачна платформа и приемане на данни	
Начало (месец): 2	**Край** (месец): 5
Цел: Изграждане на средата в AWS и единен поток в реално време на алъртите, логовете и health сигналите от всички наблюдавани среди.	
Основни задачи: 2.1. AWS акаунти, мрежа и базова сигурност, описани като код с Terraform 2.2. CI/CD конвейер за всички компоненти 2.3. Входна точка за алъртите (API Gateway, удостоверяване на източниците, парсери за основните формати) 2.4. Общ модел на събитието и разпращане към подписчиците чрез EventBridge 2.5. Свързване на четирите наблюдавани среди, включително prod 2.6. Маскиране на личните данни, преди да се подадат към LLM	
Резултати (deliverables): D2.1. Dev и staging среди на YTKI в AWS (м. 3) D2.2. Работещ CI/CD конвейер (м. 3) D2.3. Услуга за приемане на алърти (м. 4) D2.4. Модел на събитието и шина за разпращане (м. 4) D2.5. Работещи връзки към четирите среди (м. 5) D2.6. Модул за маскиране на данни (м. 5)	

Номер и име на работен пакет: РП3: Агентно разследване	
Начало (месец): 3	**Край** (месец): 8
Цел: Автоматично разследване на всеки нов алърт от специализирани агенти.	
Основни задачи: 3.1. Работен поток на разследването в Step Functions и планиращ агент, който определя какво да се провери и разпределя подзадачите 3.2. Агент за търсене и анализ на логове в Cloud Logging 3.3. Агент за проверка на health endpoint-ите 3.4. Агент за графа на зависимостите и оценка на каскадния риск 3.5. Обединяване на резултатите от агентите в хипотеза за първопричина	
Резултати (deliverables): D3.1. Работен поток с планиращ агент (м. 5) D3.2. Агент за логове (м. 6) D3.3. Агент за health проверки (м. 6) D3.4. Агент за зависимостите (м. 7) D3.5. Отчет за разследване, създаван за всеки алърт (м. 8)	

Номер и име на работен пакет: РП4: Класификация и самообучение	
Начало (месец): 4	**Край** (месец): 10
Цел: Точна класификация на инцидентите, която се подобрява с натрупването на история.	
Основни задачи: 4.1. Събиране и ръчно маркиране на минали инциденти 4.2. Класификация по трите категории с вероятност за всяка 4.3. Playwright сценарии за потвърждаване на деградация или прекъсване 4.4. Обучение и сравнение на логистична регресия и random forest 4.5. Обратна връзка от дежурните и периодично преобучение	
Резултати (deliverables): D4.1. Маркиран набор от исторически инциденти (м. 5) D4.2. Първа версия на класификатора (м. 7) D4.3. Playwright сценарии за ключовите услуги (м. 8) D4.4. Обучени модели и доклад от сравнението (м. 9) D4.5. Работещ цикъл за самообучение (м. 10)	

Номер и име на работен пакет: РП5: Интеграции и интерфейс	
Начало (месец): 6	**Край** (месец): 10
Цел: Резултатите от анализа да стигат автоматично до отговорния екип и да се виждат на едно място.	
Основни задачи: 5.1. Мапинг между алърт, компонент и отговорен екип 5.2. Автоматично създаване на събития в Splunk 5.3. Автоматично отваряне на тикети с приложен анализ 5.4. Потоци в n8n за най-честите типове инциденти 5.5. Добавяне на класификацията и хипотезите в таблото и „Ask Alerts“	
Резултати (deliverables): D5.1. Таблица за мапинг на компонентите (м. 7) D5.2. Интеграция със Splunk (м. 8) D5.3. Интеграция с тикет системата (м. 9) D5.4. Набор от n8n потоци (м. 10) D5.5. Обновени табло и „Ask Alerts“ (м. 10)	

Номер и име на работен пакет: РП6: Тестване и внедряване	
Начало (месец): 8	**Край** (месец): 12
Цел: Проверка на системата в реални условия и пускане в продукция.	
Основни задачи: 6.1. Тестване на целия поток с исторически и симулирани алърти 6.2. Тестове за натоварване (алърт бури) и отказоустойчивост 6.3. Преглед на сигурността и защитата на данните 6.4. Пилот в сенчест режим в staging и int-prod: YTKI анализира алъртите, но не отваря тикети 6.5. Изграждане на prod средата в AWS и поетапно пускане 6.6. Наблюдение на самата YTKI: аларми, табла и контролен сигнал	
Резултати (deliverables): D6.1. Отчет от функционалните тестове (м. 9) D6.2. Отчет от тестовете за натоварване (м. 10) D6.3. Доклад от прегледа на сигурността (м. 10) D6.4. Пилотен отчет с измерените метрики (м. 11) D6.5. YTKI v1.0 в продукция (м. 12) D6.6. Табла, аларми и runbook-ове за поддръжката (м. 12)	

Номер и име на работен пакет: РП7: Комуникации	
Начало (месец): 1	**Край** (месец): 12
Цел: Заинтересованите лица да знаят за проекта, да разбират резултатите и да могат да работят със системата.	
Основни задачи: 7.1. Изготвяне на комуникационен план 7.2. Информационни материали: кратко ръководство за потребителя, видео демонстрация, страница във вътрешното уики 7.3. Демонстрации пред екипите след всеки междинен резултат 7.4. Обучителни уъркшопи за дежурните екипи преди пускането в prod 7.5. Финална презентация пред ръководството	
Резултати (deliverables): D7.1. Комуникационен план (м. 1) D7.2. Ръководство за потребителя, видео и страница в уикито (м. 10) D7.3. Проведени демонстрации с кратко обобщение след всяка (м. 2–12) D7.4. Проведени уъркшопи и материали от тях (м. 11) D7.5. Финална презентация (м. 12)	

Номер и име на работен пакет: РП8: Управление на проекта	
Начало (месец): 1	**Край** (месец): 12
Цел: Проектът да приключи в срок, в рамките на бюджета и с нужното качество.	
Основни задачи: 8.1. Изготвяне на плана на проекта (обхват, график, бюджет) 8.2. Следене на напредъка чрез седмични срещи и месечни отчети 8.3. Управление на финансите, включително разходите за AWS и LLM 8.4. Управление на човешките ресурси: роли, натоварване, заместване при отсъствие 8.5. Управление на заинтересованите лица 8.6. Управление на рисковете и промените 8.7. Приключване на проекта	
Резултати (deliverables): D8.1. План за управление на проекта (м. 1) D8.2. Месечни отчети за напредъка (м. 1–12) D8.3. Бюджет и месечен отчет за разходите (м. 1–12) D8.4. Матрица на отговорностите (RACI) (м. 1) D8.5. Регистър на заинтересованите лица и план за работа с тях (м. 1) D8.6. Регистър на рисковете (от м. 1, поддържа се до края) D8.7. Финален отчет с изводи за бъдещи проекти (м. 12)	

Приложение А. Архитектура на внедряване в AWS

Това приложение описва как YTKI ще работи в AWS. То е основа за задача 1.6 (архитектура и план за внедряване), за РП2 (изграждане на средата) и за задача 6.5 (пускане в продукция).

А.1. Какво изисква системата от инфраструктурата

Изборът на архитектура следва от пет особености на YTKI.

- **Неравномерно натоварване.** В спокойни часове идват по няколко алърта, а при голям инцидент могат да дойдат стотици за минути (т.нар. алърт буря). Системата трябва да поема пиковете, без да плащаме за капацитет, който стои неизползван.
- **Дълги задачи.** Разследването на един инцидент отнема от няколко секунди до няколко минути, защото агентите чакат LLM и външни системи. Това не бива да се прави синхронно в рамките на една HTTP заявка.
- **Надеждност.** YTKI работи 24/7 и трябва да е поне толкова надеждна, колкото системите, които наблюдава. Ако тя спре, алъртите не бива да се губят.
- **Чувствителни данни.** Логовете може да съдържат лични данни, затова данните остават в ЕС и се маскират, преди да стигнат до LLM.
- **Малък екип.** Нямаме капацитет да поддържаме сървъри или Kubernetes клъстер.

А.2. Избор на подход

Разгледахме четири подхода (табл. 3).

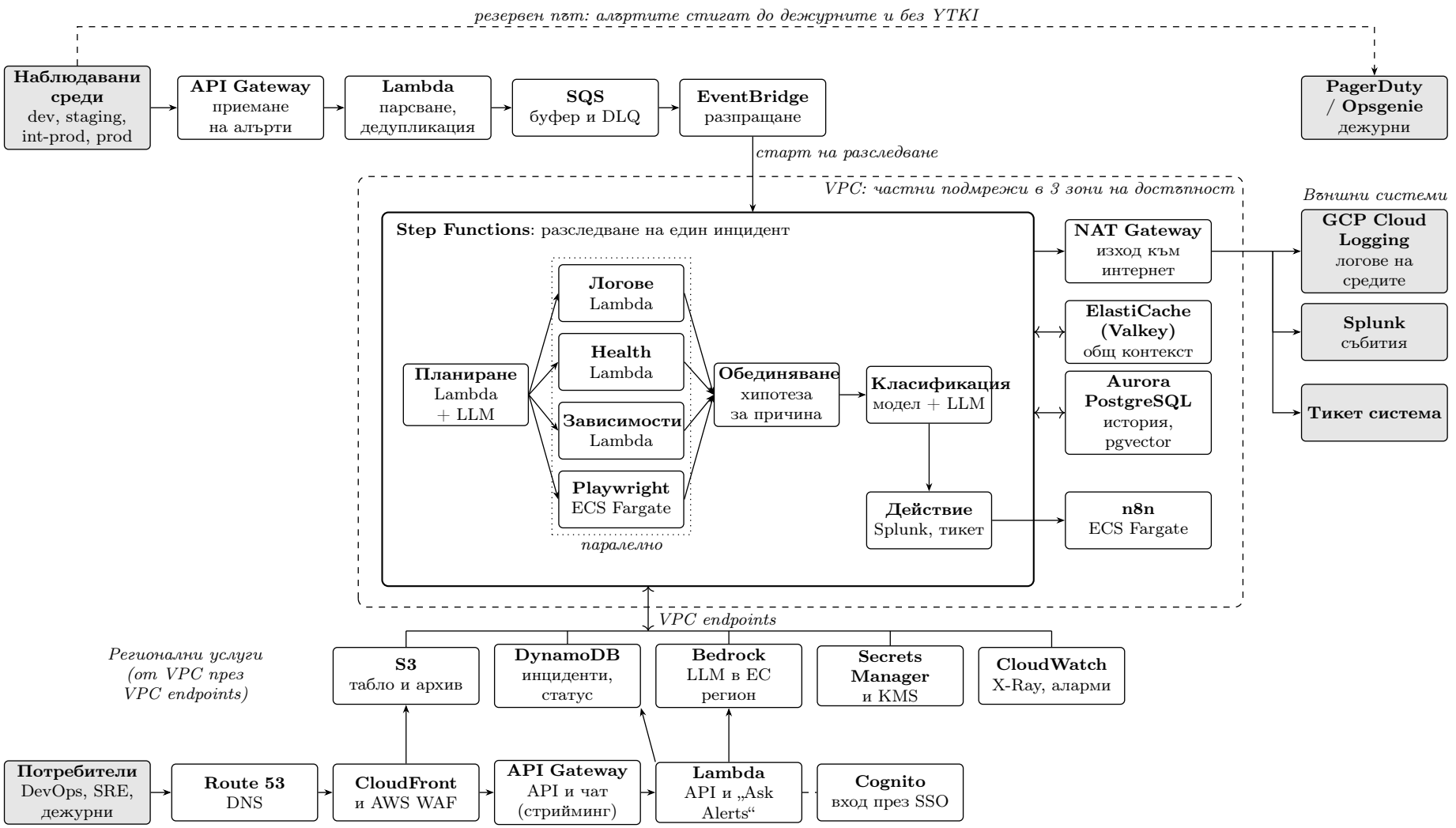
Таблица 3. Сравнение на подходите за внедряване

Подход	Предимства	Недостатъци за YTKI
Serverless (Lambda, Step Functions)	Плащаме само за реална работа; мащабира се автоматично при алърт бури; няма сървъри за поддръжка; висока наличност по подразбиране	До 15 минути на едно изпълнение на функция; студени стартове; по-трудно локално тестване
Контейнери в ECS Fargate	Няма ограничение във времето; подходящо за браузъри и постоянно работещи услуги	Плащаме за задачите, докато работят, дори без натоварване; мащабирането се настройва ръчно
Kubernetes (EKS)	Голяма гъвкавост и преносимост между облаци	Висока сложност и постоянна цена на клъстера; нужен е човек само за поддръжката му
Виртуални машини (EC2)	Пълен контрол върху средата	Обновяване на ОС, ръчна висока наличност и мащабиране

Избираме **serverless като основа** и ECS Fargate само за двата компонента, които не пасват на Lambda: Playwright сценариите (нужен е цял браузър) и n8n (постоянно работеща услуга). Разследването не се изпълнява синхронно зад API-то. Входната точка само приема алърта, връща отговор веднага и го оставя в опашка, а самото разследване се оркестрира от Step Functions. Така времето за отговор към системите за мониторинг не зависи от това колко бавно отговаря LLM.

А.3. Обща схема

Схемата е показана на фиг. 3. Системата има три входа: алъртите от наблюдаваните среди (горе), потребителите на таблото и чата (долу) и резервния път, по който алъртите стигат до дежурните и без YTKI.



Фигура 3. Архитектура на YTKI в AWS. Сивите кутии са външни системи. Step Functions е регионална услуга; в VPC работят функциите и задачите, които тя стартира.

А.4. Път на един алърт през системата

1. Системата за мониторинг (напр. Alertmanager или Grafana) изпраща алърта едновременно към PagerDuty или Opsgenie и към входната точка на YTKI. Така дежурните получават алърта дори ако YTKI не работи.
2. API Gateway проверява токена на източника и ограничава броя на заявките. Lambda функцията парсва алърта, превръща го в общия модел на събитието и изчислява неговия отпечатък (fingerprint). Ако същият алърт вече е приет в последните минути, той се отбелязва като дубликат и не се разследва отново.
3. Алъртът влиза в SQS опашка. Тя поема алърт бурите, а съобщенията, които не могат да се обработят, отиват в отделна опашка (DLQ) за преглед и повторно изпращане.
4. EventBridge разпраща събитието към абонатите. Това е абонаментният модел от фиг. 1 на Домашна работа 1: правило стартира разследване, друго записва събитието в архива, а в бъдеще може да се добавят нови абонати без промяна на кода.
5. Step Functions изпълнява разследването. Планиращият агент решава какво да се провери, след което агентите за логове, health проверки, зависимости и Playwright работят паралелно. Междинните им резултати се записват в общ контекст в ElastiCache.
6. Резултатите се обединяват в хипотеза за първопричина. Класификаторът определя дали става дума за фалшив алърт, деградация или прекъсване, с вероятност за всяка категория. Историята на подобни минали инциденти се търси в Aurora (pgvector).
7. При деградация или прекъсване системата създава събитие в Splunk, отваря тикет към отговорния екип и при нужда стартира поток в n8n. Отчетът за разследването се записва в DynamoDB и S3 и се появява в таблото.
8. Когато дежурният потвърди или поправи класификацията, отговорът му се записва като нов пример за обучение (РП4).

А.5. Компоненти

Таблица 4. Компоненти на системата и роля на всеки от тях

Слой	**AWS услуга**	**За какво я използваме**
DNS	Route 53	Домейните на таблото и на входната точка за алъртите
Край на мрежата	CloudFront и AWS WAF	Доставят таблото, филтрират злонамерен трафик и ограничават броя на заявките
Удостоверяване	Cognito	Вход през корпоративния доставчик на идентичност (SSO), JWT токени за API-то
API	API Gateway (REST, регионален)	Две API-та: за приемане на алърти и за таблото и чата. Отговорите в чата се стриймват към потребителя
Буфер	SQS с DLQ	Поема алърт бурите и пази съобщенията, които не са обработени
Разпращане	EventBridge	Абонаментният модел: правила насочват събитията към абонатите; архив за повторно изпращане
Оркестрация	Step Functions (Standard)	Слоевете Planning, Execution, Aggregation, Classification и Operation; паралелно изпълнение на агентите; история на всяко разследване за одит
Изчисления	Lambda (в VPC)	Агентите, обединяването, класификацията и интеграциите
Изчисления	ECS Fargate	Playwright сценариите и n8n (зад вътрешен ALB)
LLM	Amazon Bedrock	Езиковите модели, използвани от ЕС регион; данните не излизат от AWS

Слой	AWS услуга	За какво я използваме
Горещи данни	ElastiCache Serverless (Valkey)	Общ контекст на агентите, прозорец за дедупликация, ограничаване на заявките към външни API
Състояние	DynamoDB	Инциденти, алърти и статус на разследванията; данни за таблото
История	Aurora PostgreSQL Serverless v2 с pgvector	История на инцидентите, търсене на сходни случаи, мапинг на компонентите
Архив	S3	Файловете на таблото, сурови събития, отчети и данни за обучение на моделите
Тайни	Secrets Manager и KMS	Токените за Splunk, тикет системата и GCP; криптиране на данните
Наблюдение	CloudWatch, X-Ray, OpenTelemetry	Логове, метрики и трасировки на самата YTKI; аларми

A.6. Мрежа и сигурност

Регион. Цялата система работи в eu-central-1 (Франкфурт). Данните остават в ЕС, което улеснява спазването на GDPR.

VPC. Използваме три зони на достъпност. Lambda функциите, задачите във Fargate, ElastiCache и Aurora са в частни подмрежи без публични адреси. Публичните подмрежи съдържат само NAT Gateway. Lambda функциите се свързват с VPC чрез мрежови интерфейси (ENI), които AWS създава при конфигуриране на функцията, затова подмрежите трябва да имат достатъчно свободни адреси. Security групите разрешават достъп до Aurora и ElastiCache само от функциите и задачите на YTKI.

Изходящ трафик. Към S3 и DynamoDB се стига през безплатни gateway endpoints, а към Bedrock, SQS, Step Functions, Secrets Manager, KMS, CloudWatch и ECR през interface endpoints. Така трафикът към AWS услугите не минава през NAT. NAT Gateway остава само за външни системи като GCP, Splunk и тикет системата. Ако тези системи поддържат IPv6, NAT може да се замени с egress-only internet gateway, който няма почасова такса.

Входна точка за алъртите. Всеки източник получава отделен токен, който се проверява от Lambda authorizer. Към API-то е закачен WAF с ограничение на броя заявки. Като следваща стъпка може да се добави mTLS.

Достъп до GCP. Агентът за логове чете от Cloud Logging чрез Workload Identity Federation: IAM ролята на функцията се разменя за краткотраен токен в GCP, без да пазим ключове.

Права. Всяка функция има собствена IAM роля само с най-необходимите права. CI/CD конвейерът влиза в AWS чрез OIDC, без постоянни ключове. CloudTrail, GuardDuty и Security Hub са включени във всички акаунти.

Данни. Всичко се криптира с ключове в KMS. Личните данни се маскират преди подаване към Bedrock (задача 2.6), а логовете на самата YTKI се пазят ограничено време.

A.7. Среди и стратегии за внедряване

Акаунти. Чрез AWS Organizations имаме отделни акаунти за инструментите (CI/CD, ECR), за dev, staging и prod на YTKI и за централизиран архив на логовете. Средите на YTKI не съвпадат със средите, които тя наблюдава: prod версията на YTKI приема алърти от всички четири наблюдавани среди, а dev и staging версиите получават копия или записани исторически алърти.

Инфраструктура като код. Цялата инфраструктура е описана с Terraform в модули (мрежа,

данни, изчисления, край на мрежата). Едни и същи модули се използват за всички акаунти, с различни параметри. Избираме Terraform, защото с него може да опишем и частта в GCP (Workload Identity Federation), а не само тази в AWS.

Конвейер. GitHub Actions с OIDC към AWS. Стъпките са: проверки и unit тестове, изграждане на пакетите за Lambda, образите за ECR и файловете на таблото, `terraform plan` с ревю в pull request, внедряване в dev, интеграционни тестове с повторно изпращане на исторически алърти, внедряване в staging, ръчно одобрение и внедряване в prod.

Таблица 5. Стратегии за пускане на нови версии

Компонент	Стратегия	Връщане назад
Lambda функции	Версии и псевдоними; CodeDeploy пренасочва първо 10% от трафика и следи алармите	Автоматично, ако аларма се задейства
Step Functions	Версии и псевдоними с постепенно пренасочване между старата и новата версия	Псевдонимът се връща към старата версия
Fargate (n8n, Playwright)	Вградения blue/green в ECS	Трафикът се връща към старите задачи
Табло (S3 и CloudFront)	Новата версия се качва в отделна папка и CloudFront се пренасочва след проверка	Пренасочване към предишната папка
Схема на Aurora	Миграции в две стъпки (expand/contract): първо се добавя новото, старото се маха в следваща версия	Старият код продължава да работи с новата схема
Класификатор и промптове	Сенчест режим: новата версия работи паралелно със старата, без да предприема действия; резултатите се сравняват	Новата версия просто не се активира
Инфраструктура	`terraform plan` в pull request, ревю, прилагане само през конвейера	Прилагане на предишната версия на кода

А.8. Надеждност и наблюдение на самата система

YTKI не е в критичния път. Алъртите стигат до дежурните по стария път, а YTKI само ги допълва с анализ. Ако тя спре, губим анализа, но не и алъртите.

Нищо не се губи. SQS опашката с DLQ и архивът в EventBridge позволяват алъртите да се изпратят повторно. Отпечатъкът на алърта служи като ключ за идемпотентност, затова повторното изпращане не създава дублирани тикети.

Контролен сигнал. На всеки пет минути през системата минава изкуствен алърт. Ако за него не се появи отчет навреме, CloudWatch вдига аларма директно към PagerDuty.

Отказ на LLM. При грешки или ограничения от Bedrock Step Functions повтаря заявката с нарастващо изчакване, а при продължителен проблем алъртът минава без анализ, с ясна бележка в таблото.

Резервни копия. DynamoDB е с възстановяване към точен момент, Aurora прави автоматични резервни копия, а S3 пази версии. AWS Backup копира данните и в друг ЕС регион. Предлагаме цел за загуба на данни до 15 минути (RPO) и за възстановяване до 4 часа (RTO).

Наблюдение. Всяко разследване се проследява с OpenTelemetry от приемането до действието. Таблата в CloudWatch показват време за разследване, грешки, изразходвани токени и цена на един инцидент.

А.9. Разходи

Променливите разходи са за Lambda, Step Functions, API Gateway и най-вече за токените в Bedrock. Постоянните разходи са за NAT Gateway и interface endpoints (таксуват се на час за всяка зона), минималния капацитет на Aurora и ElastiCache и постоянно работещата задача на n8n. За да ги намалим, в dev средата използваме една зона и един NAT, а Aurora спира, когато не се използва. Дубликатите се отсяват преди LLM, за по-простите стъпки (планиране, класификация) се ползва по-малък модел, а AWS Budgets и Cost Anomaly Detection следят сметката. Точната оценка ще бъде част от т. 2.3 (разпределение на ресурсите).

А.10. Разгледани алтернативи

Lambda durable functions вместо Step Functions. Позволяват многостъпкови процеси директно в кода, без отделна услуга. Предпочитаме Step Functions, защото дава визуална история на всяко разследване, която е полезна за одит, и истинско паралелно изпълнение на агентите.

Amazon Bedrock AgentCore Runtime за агентите. Предлага изолирани сесии с продължителност до няколко часа. Подходящ е, ако „Ask Alerts“ прерасне в дълги интерактивни разследвания, и е възможна следваща стъпка след v1.0.

Само DynamoDB, без ElastiCache. Ще намали броя на компонентите. Запазваме ElastiCache заради бързия общ контекст и ограничаването на заявките при алърт бури, но в първата версия тя може да се пропусне и да се добави, ако измерванията покажат нужда.